

Tuplas e dicionários

As linguagens de programação possuem estruturas poderosas para abstrações de dados essenciais ao desenvolvimento de aplicações robustas e adequadas ao contexto de uso. Em Python, não é diferente: a linguagem oferece diversas estruturas que permitem adaptar soluções de tecnologia da informação às necessidades específicas do projeto. Conforme discutido por Forbellone e Eberspächer (2022), a organização e a manipulação eficiente de dados são fundamentais para a construção de algoritmos bem-estruturados, sendo as tuplas e os dicionários duas das principais ferramentas para esse fim.

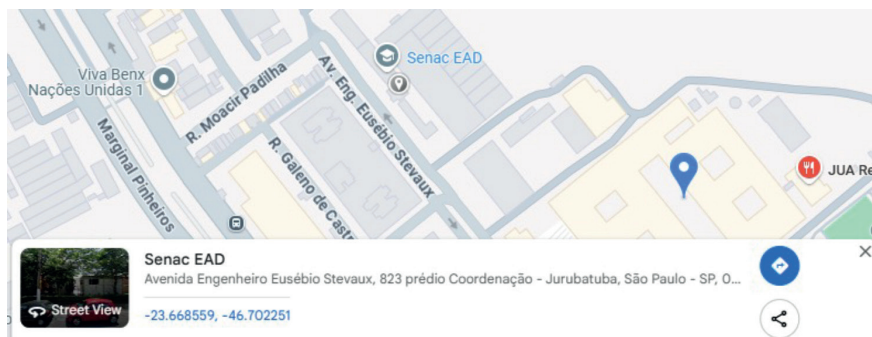
Neste capítulo, serão abordadas as estruturas de tuplas e dicionários, comuns em linguagens multifuncionais, como a Python, pois oferecem diferentes formas de representação de dados e flexibilidade em sua manipulação, permitindo a adaptação a distintos contextos de desenvolvimento (Guilhon *et al.*, 2022). No primeiro subcapítulo, será tratado o conceito de tupla, enfatizando sua característica posicional e imutável. Em seguida, serão abordados o uso de dicionários e sua capacidade de armazenar dados de maneira rotulada, facilitando o processo de

recuperação e organização, como pontua Lambert (2022). Por fim, serão discutidas situações específicas do uso combinado de dicionários e tuplas em conjunto com funções, demonstrando sua aplicabilidade prática na construção de soluções eficientes.

1 Tuplas

No mundo da computação e da matemática, temos algumas representações para as quais a ordem e a imutabilidade são garantias para a confiabilidade de um dado. Um primeiro exemplo são os pontos de sistemas cartesianos (bidimensionais ou tridimensionais), aplicados nas diferentes representações de dados. A figura 1 mostra um exemplo de aplicação da representação da latitude e longitude em um mapa, na qual é imprescindível a ordem de apresentação dos dados na estrutura.

Figura 1 – Exemplo do uso de coordenadas geográficas em um mapa



Outro exemplo que envolve nosso cotidiano está relacionado aos sistemas de cores utilizados no meios digitais e impressos. No meio digital, o sistema de cor utilizado é o RGB (representação das cores red, green e blue – vermelho, verde e azul), e no meio impresso é utilizado o padrão CMKY (representação das cores cyan, magenta, yellow e key – ciano, magenta, amarelo e preto). É importante também reforçar que a ordem de representação dos dados também é imprescindível. O quadro 1 apresenta alguns exemplos de combinações do padrão RGB e CYMK.

Quadro 1 – Exemplo da representação do padrão RGB e CMYK

COR	RGB	CMYK
Vermelho	RGB(255, 0, 0)	CMYK(0, 100, 100, 0)
Verde	RGB(0, 255, 0)	CMYK(100, 0, 100, 0)
Azul	RGB(0, 0, 255)	CMYK(100, 100, 0, 0)
Amarelo	RGB(255, 255, 0)	CMYK(0, 0, 100, 0)
Ciano	RGB(0, 255, 255)	CMYK(100, 0, 0, 0)
Magenta	RGB(255, 0, 255)	CMYK(0, 100, 0, 0)
Branco	RGB(255, 255, 255)	CMYK(0, 0, 0, 0)
Preto	RGB(0, 0, 0)	CMYK(0, 0, 0, 100)
Cinza	RGB(128, 128, 128)	CMYK(0, 0, 0, 50)

Na linguagem de programação Python, as tuplas são estruturas que melhor representam os exemplos apresentados, dada sua imutabilidade; ou seja, *os dados não podem ser alterados depois de sua criação*. Para criar um tupla em Python, a seguinte sintaxe deve ser utilizada:

```
minha_tupla = (1, 2, 3)
```

Em que cada elemento da tupla deve ser atribuído na ordem adequada para o futuro tratamento dos dados.

Corroborando com nossos exemplos, podemos representar coordenadas geográficas e cores utilizando tuplas, conforme segue.

```
#coordenada geográfica
lat_long_senac = (-23.668559, -46.702251)
#rgb
vermelho_rgb = (255, 0, 0)
#cmky
amarelo_cmyk = (0, 0, 100, 0)
```

Para acessar o elemento de uma tupla, basta utilizar seu índice, começando por zero até o n-ésimo elemento menos 1. Por exemplo: se a tupla possui 3 elementos, os índices serão rotulados de 0 a 2. Observe o exemplo 1.

Exemplo 1

```
print(vermelho_rgb[0]) # Saída: 255
print(amarelo_cmyk[2]) # Saída: 100
```

Outro ponto importante é entender que tuplas também permitem diferentes tipos de dados, como mostra o exemplo 2.

Exemplo 2

```
info_cor = ("Vermelho", (255, 0, 0), "#FF0000", True)
print(info_cor)
# Saída: ('Vermelho', (255, 0, 0), '#FF0000', True)
```

No exemplo 2, "Vermelho" é uma string, (255, 0, 0) é uma tupla aninhada representando RGB, '#FF0000' é a representação hexadecimal da cor e True indica que essa cor é considerada "quente".

As tuplas em Python possuem uma opção de desempacotamento, o qual seria o processo simplificado de atribuição dos dados da tupla a variáveis individuais. Observe o exemplo 3.

Exemplo 3

```
r, g, b = vermelho_rgb
print(f"R: {r}, G: {g}, B: {b}") # Saída: R: 255, G: 0, B: 0
```

Por fim, é importante reforçar que a imutabilidade de tuplas em Python é uma premissa da linguagem; ou seja, caso seja feita uma tentativa de armazenar um novo valor em um elemento de uma tupla, um erro será gerado.

```
vermelho_rgb[0] = 200 # Isso resultará em um erro!
```



IMPORTANTE

As tuplas desempenham um papel fundamental na matemática e na computação em Python, pois oferecem uma maneira eficiente e estruturada de representar conjuntos imutáveis de valores. Sua imutabilidade garante segurança em operações que exigem dados fixos, como coordenadas cartesianas, vetores, pontos em gráficos, constantes físicas e representações de cores nos modelos RGB e CMYK, em que cada cor pode ser armazenada como uma tupla de três ou quatro valores, respectivamente. Além disso, a capacidade das tuplas de armazenar diferentes tipos de dados as torna ideais para representar estruturas heterogêneas, como registros de banco de dados, informações de configuração e agrupamentos de valores matemáticos em álgebra linear e geometria computacional. Seu desempenho superior em relação às listas, devido à otimização interna de armazenamento, faz das tuplas uma escolha eficiente para aplicações que exigem integridade dos dados e processamento rápido.

2 Dicionários

As linguagens de programação, ao longo de sua evolução, se adaptaram às necessidades de flexibilização no tratamento de dados de maneira a melhorar a produtividade na proposição de problemas na tecnologia da informação. Principalmente com o advento de padrões de permitem o intercâmbio de dados tais como o JSON e o XML, foram necessárias adaptações às linguagens de programação para trabalhar com esses tipos de dados com maior facilidade. As figuras 2 e 3 mostram os dados de uma pessoa em formato JSON e XML, respectivamente.

Figura 2 – Dados pessoais em formato JSON

```
{
  "pessoa": {
    "nome": "João da Silva",
    "idade": 35,
    "email": "joao.silva@email.com",
    "telefone": "+55 11 99999-8888",
    "endereco": {
      "rua": "Av. Paulista",
      "numero": 1234,
      "cidade": "São Paulo",
      "estado": "SP",
      "cep": "01311-200"
    },
    "documentos": {
      "cpf": "123.456.789-00",
      "rg": "12.345.678-9"
    }
  }
}
```

Figura 3 – Dados pessoais em formato XML

```
<pessoa>
  <nome>João da Silva</nome>
  <idade>35</idade>
  <email>joao.silva@email.com</email>
  <telefone>+55 11 99999-8888</telefone>
  <endereco>
    <rua>Av. Paulista</rua>
    <numero>1234</numero>
    <cidade>São Paulo</cidade>
    <estado>SP</estado>
    <cep>01311-200</cep>
  </endereco>
  <documentos>
    <cpf>123.456.789-00</cpf>
    <rg>12.345.678-9</rg>
  </documentos>
</pessoa>
```

Contudo, podem também existir outros formatos específicos além do JSON e XML que permitem esta flexibilidade de porte de diferentes tipos de dados. Nesse caso, como a linguagem de programação pode trabalhar com a manipulação desses dados como uma estrutura única? A linguagem de programação Python soluciona esse tratamento através dos dicionários, sendo estruturas que têm por característica o armazenamento de dados em padrão de chave-valor. Eles permitem acesso rápido aos valores por meio de suas chaves, funcionando de forma semelhante a um mapa ou uma tabela de consulta. Para criar um dicionário, a sintaxe básica consiste no uso do `{}` ou a função `dict()`, como apresentado no exemplo 4.

Exemplo 4

```
# Criando um dicionário de uma pessoa
pessoa = {
    "nome": "João da Silva",
    "idade": 35,
    "email": "joao.silva@email.com",
    "telefone": "+55 11 99999-8888"
}

print(pessoa)
#saida
# {'nome': 'João da Silva', 'idade': 35, 'email': 'joao.silva@email.com', 'telefone': '+55 11 99999-8888'}
```

Nos exemplos das figuras 2 e 3, os respectivos dados podem ser representados em um dicionário em Python da seguinte maneira.

Exemplo 5

```
pessoa = {
    "nome": "João da Silva",
    "idade": 35,
    "email": "joao.silva@email.com",
    "telefone": "+55 11 99999-8888",
    "endereco": {
```

```

        "rua": "Av. Paulista",
        "numero": 1234,
        "cidade": "São Paulo",
        "estado": "SP",
        "cep": "01311-200"
    },
    "documentos": {
        "cpf": "123.456.789-00",
        "rg": "12.345.678-9"
    }
}

# Exemplo de acesso aos dados
print(pessoa["nome"]) # Saída: João da Silva
print(pessoa["endereço"]["cidade"]) # Saída: São Paulo
print(pessoa["documentos"]["cpf"]) # Saída: 123.456.789-00

```

É importante observar ao final do exemplo 5 o acesso aos valores de um dicionário, bastando informar a chave para receber um valor desejado. Outro ponto de observação é que dicionários também podem armazenar outros dicionários (dados de endereço e dados de documentos). Para acessar esses dados, basta mencionar as chaves em ordem de profundidade de acesso sempre utilizando `[]`.

Dicionários possuem elementos mutáveis, ou seja, passíveis de modificação ao longo de seu uso. A seguir, são apresentados exemplos de atribuição de novos dados a um dicionário, bastando informar a chave para a localização exata do ponto de alteração.

Exemplo 6

```

pessoa["idade"] = 36 # Atualizando um valor existente
pessoa["cidade"] = "São Paulo" # Adicionando uma nova chave

```

Para remover itens de um dicionário, deve ser utilizado o comando `del`, especificando a chave que pretende ser removida, como apresentam os exemplos a seguir. Outra opção apresentada é a remoção de uma chave e o retorno do valor através do comando `pop`.

Exemplo 7

```
del pessoa["telefone"] # Remove uma chave específica
print(pessoa)

telefone = pessoa.pop("email") # Remove e retorna o valor
associado
print(telefone) # Saída: joao.silva@email.com
```

Outro aspecto importante no que diz respeito aos dicionários é a técnica de iteração sobre os elementos. Para cada iteração, é acessado o item no qual uma tupla pode ser acessada, na qual o primeiro elemento é a chave e o segundo elemento é o valor, conforme o exemplo 8 a seguir.

Exemplo 8

```
# Percorrendo chaves e valores
for chave, valor in pessoa.items():
    print(f"{chave}: {valor}")
```

O dicionário possui em sua estrutura métodos nativos (funções implementadas para a estrutura específica) que facilitam a manipulação, tal como os métodos `keys()` (retorna todas as chaves), `values()` (retornam todos os valores) e `items()` (como apresentado no exemplo 9, retorna todos os itens).

Exemplo 9

```
print(pessoa.keys()) # Retorna todas as chaves
print(pessoa.values()) # Retorna todos os valores
print(pessoa.items()) # Retorna pares chave-valor
```

Uma opção importante que possibilita à pessoa desenvolvedora otimizar seu tempo é o método de verificação da existência de uma chave através do método `in`, como segue. O código apresentado no exemplo 10 verifica se a chave `email` está presente no dicionário `pessoa`.

Exemplo 10

```
if "email" in pessoa:
    print("E-mail cadastrado:", pessoa["contato"]["email"])
```



IMPORTANTE

Os dicionários em Python desempenham um papel crucial na manipulação de dados intercambiáveis, como aqueles representados em JSON e XML, facilitando a comunicação entre sistemas, APIs e bancos de dados. Sua estrutura baseada em pares chave-valor permite acesso eficiente às informações, tornando-os ideais para converter, armazenar e processar dados estruturados de forma dinâmica e flexível. Como o JSON é nativamente compatível com dicionários, a conversão entre esses formatos é simples, permitindo que aplicações web e serviços distribuídos troquem informações de maneira padronizada. Além disso, a manipulação de XML pode ser feita de forma eficiente utilizando dicionários, permitindo extrair, modificar e serializar dados para diversas aplicações. Essa capacidade torna os dicionários essenciais para o desenvolvimento de software moderno, garantindo interoperabilidade, organização e eficiência na manipulação de grandes volumes de dados.

3 Tuplas e dicionários em funções

As funções em Python podem receber tuplas e dicionários como argumentos e podem retorná-los como resultado. Isso é útil para manipular múltiplos valores de forma eficiente e organizada.

Tuplas são úteis quando uma função precisa receber um conjunto fixo de valores que não devem ser modificados. Perceba no exemplo 11 a passagem da tupla como argumento de uma função, destacando que ela tem a garantia de que os dados não podem ser modificados e consequentemente a função não pode modificar seus valores internos.

Exemplo 11

```
def exibir_dados_pessoais(dados):
    nome, idade, cidade = dados # Desempacotamento da tupla
    print(f"Nome: {nome}, Idade: {idade}, Cidade: {cidade}")

# Criando uma tupla com dados
pessoa = ("João da Silva", 35, "São Paulo")

# Passando a tupla como argumento
exibir_dados_pessoais(pessoa)

# saída
# Nome: João da Silva, Idade: 35, Cidade: São Paulo
```

Funções também pode retornar tuplas em casos em que seja necessário o retorno de um conjunto de valores em uma única estrutura imutável sem o uso de listas ou dicionários, como mostra o exemplo 12.

Exemplo 12

```
def calcular_idade_futura(idade_atual, anos):
    idade_futura = idade_atual + anos
    return idade_atual, idade_futura # Retorno como tupla

idade_atual, idade_futura = calcular_idade_futura(35, 5)
print(f"Idade atual: {idade_atual}, Idade em 5 anos: {idade_futura}")

#saída
# Idade atual: 35, Idade em 5 anos: 40
```

Quando se trata do uso de dicionários na passagem de valores ou retornos em funções, devem ser tomados os devidos cuidados, principalmente pela flexibilidade da estrutura. A passagem de um dicionário como um argumento de uma função é nativamente por referência, ou seja, a estrutura passada é manipulada na origem em que foi criada, sem a realização de uma cópia. O código-fonte, no exemplo 13 a seguir, mostra o uso de uma função que facilita a visualização dos dados de um dicionário.

Exemplo 13

```
def exibir_dados_pessoais(dados):
    print(f"Nome: {dados['nome']}, Idade: {dados['idade']},
    Cidade: {dados['cidade']}")

pessoa = {
    "nome": "João da Silva",
    "idade": 35,
    "cidade": "São Paulo"
}

exibir_dados_pessoais(pessoa)
#saida
#Nome: João da Silva, Idade: 35, Cidade: São Paulo
```

Reforçando a mutabilidade dos dicionários, o exemplo abaixo faz a inclusão de um par-chave e valor em um dicionário passado como argumento de uma função. Observe que esse dicionário é modificado em sua origem, como demonstra o exemplo 14. A função modificou diretamente o dicionário original, pois foi passado por referência.

Exemplo 14

```
def adicionar_email(pessoa):
    pessoa["email"] = "joao.silva@email.com"

dados_pessoais = {"nome": "João da Silva", "idade": 35}
adicionar_email(dados_pessoais)

print(dados_pessoais)

#saida
# {'nome': 'João da Silva', 'idade': 35, 'email': 'joao.silva@email.com'}
```

Quando a intenção não é a manipulação do dicionário por referência e, sim, ter uma cópia para ao final retornar um novo dicionário, é importante o uso do comando `copy()`. O exemplo 15 faz uma cópia simples de

um dicionário e retorna um novo, desvinculando o novo dicionário da origem do anterior.

Exemplo 15

```
def adicionar_email(pessoa):  
    nova_pessoa = pessoa.copy()  
    nova_pessoa["email"] = "joao.silva@email.com"  
    return nova_pessoa  
  
dados_pessoais = {"nome": "João da Silva", "idade": 35}  
dados_modificados = adicionar_email(dados_pessoais)  
  
print(dados_pessoais) # Original não é alterado  
print(dados_modificados) # Novo dicionário com a modificação
```



PARA SABER MAIS

Existem mais funcionalidades dos dicionários que podem ser exploradas! O `deepcopy()`, presente no módulo `copy`, é utilizado para criar cópias independentes de objetos complexos, garantindo que estruturas aninhadas sejam replicadas sem manter referências ao original, evitando modificações indesejadas. Já o `**kwargs` permite que funções recebam um número variável de argumentos nomeados em formato de dicionário, tornando-as mais flexíveis e reutilizáveis. Você pode pesquisar por ambas as funcionalidades em sua ferramenta de busca para aprofundar ainda mais seus estudos sobre dicionários.

Considerações finais

Neste capítulo, exploramos as tuplas e os dicionários como estruturas fundamentais da linguagem Python, destacando as características, os usos e as vantagens na manipulação de dados. As tuplas, com sua imutabilidade e organização posicional, se mostraram ideais para representar dados fixos, como coordenadas e modelos de cores. Já os

dicionários, por serem estruturas mutáveis e baseadas em pares chave-valor, se destacaram pela flexibilidade no armazenamento e recuperação de informações, sendo amplamente utilizados na manipulação de formatos como JSON e XML. Além disso, abordamos a interação dessas estruturas com funções, demonstrando a passagem de argumentos e o retorno de valores, bem como os impactos da imutabilidade das tuplas e da passagem por referência dos dicionários.

Referências

FORBELLONE, André Luiz Villar; EBERSPÄCHER, Henri Frederico. **Lógica de programação**: a construção de algoritmos e estruturas de dados com aplicações em Python. 4. ed. São Paulo: Bookman, 2022. *E-book*.

GUILHON, André *et al.* (org.). **Jornada Python**: uma jornada imersiva na aplicabilidade de uma das mais poderosas linguagens de programação do mundo. Rio de Janeiro, RJ: Brasport, 2022. *E-book*.

LAMBERT, Kenneth A. **Fundamentos de Python**: estruturas de dados. Cengage Learning Brasil, 2022. *E-book*.